

Les bases



The PHP logo, featuring the letters 'php' in a white, lowercase, sans-serif font, set against a large, light purple oval background.

FORMATION PHP - LES BASES

PHP

Créer des pages web dynamiques, pas à pas.

DanielCraft - Livre débutant - Pack Web

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [C'est quoi PHP ?](#)

- [Le langage en une phrase](#)
- [Pourquoi apprendre PHP ?](#)
- [A quoi ca ressemble ?](#)
- [Petite histoire](#)
- [En vrai](#)
- [A retenir](#)

2. [Installer PHP](#)

- [Ce qu'il te faut](#)
- [Option 1 : serveur integre \(recommande pour debuter\)](#)
- [Option 2 : XAMPP](#)
- [Installer VS Code](#)
- [Petite histoire](#)
- [A retenir](#)

3. [Premier programme](#)

- [Hello World en PHP](#)
- [Les balises PHP](#)
- [echo vs print](#)
- [Les commentaires](#)
- [Petite histoire](#)
- [A retenir](#)

4. [Les variables](#)

- [Declarer une variable](#)
- [Regles de nommage](#)
- [Affichage](#)
- [Constantes](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [A retenir](#)

5. [Les types de donnees](#)

- [Les types de base](#)
- [Verifier un type](#)
- [Conversion](#)
- [Operations](#)
- [Petite histoire](#)
- [A retenir](#)

6. [Les conditions](#)

- [if / else](#)
- [if / elseif / else](#)
- [Operateur ternaire](#)

- [Null coalescing](#)
- [Switch](#)
- [Match \(PHP 8+\)](#)
- [Petite histoire](#)
- [A retenir](#)

7. [Les boucles](#)

- [for](#)
- [foreach](#)
- [while](#)
- [do-while](#)
- [break et continue](#)
- [Petite histoire](#)
- [A retenir](#)

8. [Les fonctions](#)

- [Déclarer une fonction](#)
- [Retourner une valeur](#)
- [Paramètres optionnels](#)
- [Types de retour \(PHP 8+\)](#)
- [Fonctions anonymes et flèches](#)
- [Petite histoire](#)
- [A retenir](#)

9. [Les tableaux](#)

- [Tableaux indexés](#)
- [Ajouter et supprimer](#)
- [Tableaux associatifs](#)
- [Parcourir](#)
- [Fonctions utiles](#)
- [Petite histoire](#)
- [A retenir](#)

10. [Classes et objets](#)

- [C'est quoi une classe ?](#)
- [Constructeur promu \(PHP 8+\)](#)
- [Visibilité](#)
- [Héritage](#)
- [Petite histoire](#)
- [A retenir](#)

11. [Les formulaires](#)

- [Recevoir des données](#)
- [GET vs POST](#)
- [Valider les entrées](#)
- [Sécuriser les sorties](#)
- [Petite histoire](#)
- [A retenir](#)

12. Gérer les erreurs

- [Les exceptions](#)
- [Erreurs courantes](#)
- [Configurer le reporting](#)
- [Lever une exception](#)
- [Petite histoire](#)
- [À retenir](#)

13. Mini-projet : page de contacts

- [L'objectif](#)
- [Structure](#)
- [Ce que tu apprends](#)
- [À retenir](#)

14. Ce qu'il faut retenir

- [Les briques essentielles](#)
- [La méthode](#)
- [Ce qui vient après](#)
- [À retenir](#)

15. Atelier : variables et types

- [Exercice 1 : carte d'identité](#)
- [Exercice 2 : convertisseur EUR -> USD](#)
- [Exercice 3 : comparaison stricte](#)
- [Défi : température C -> F](#)
- [À retenir](#)

16. Atelier : fonctions

- [Exercice 1 : salutation](#)
- [Exercice 2 : moyenne](#)
- [Exercice 3 : mot de passe valide](#)
- [Exercice 4 : fonction flèche](#)
- [À retenir](#)

17. Atelier : tableaux

- [Exercice 1 : filtrer les pairs](#)
- [Exercice 2 : compteur de mots](#)
- [Exercice 3 : transformer un tableau](#)
- [Exercice 4 : panier d'achat](#)
- [À retenir](#)

18. Erreurs classiques en PHP

- [1. Oublier le \\$ devant une variable](#)
- [2. Confondre = et ==](#)
- [3. Type juggling surprenant](#)
- [4. Oublier htmlspecialchars](#)
- [5. Tableau non initialisé](#)
- [6. Point-virgule oublié](#)

- [A retenir](#)

19. [Bonnes pratiques](#)

- [Style de code](#)
- [Typage strict](#)
- [Securite](#)
- [Organisation](#)
- [Outils](#)
- [A retenir](#)

20. [Quiz PHP - Les bases](#)

- [Teste tes connaissances](#)
- [Reponses](#)

21. [Bravo !](#)

- [Tu as termine PHP - Les bases](#)
- [Ce que tu as construit](#)
- [La suite avec DanielCraft](#)
- [Un dernier conseil](#)

C'est quoi PHP ?



PHP : 77% du web.

Le langage en une phrase

PHP est un langage de programmation cote serveur, cree en 1995 par Rasmus Lerdorf. Il genere des pages web dynamiques : formulaires, bases de donnees, sessions, API. Environ 77% des sites web utilisent PHP (dont WordPress, Wikipedia, Facebook a ses debuts).

Pourquoi apprendre PHP ?

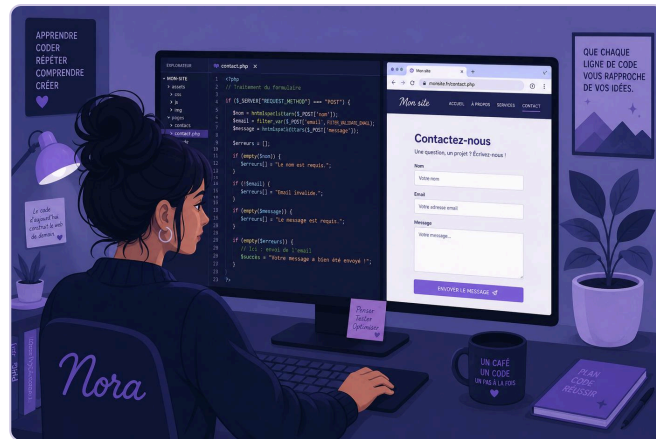
PHP reste le langage serveur le plus deploye au monde. WordPress (43% du web), Laravel, Symfony : l'ecosysteme est immense. Les offres d'emploi PHP sont nombreuses, surtout en agence web et e-commerce.

> **Astuce DanielCraft** - PHP est ideal pour commencer le backend. Tu vois le resultat directement dans le navigateur.

Nora installe PHP et XAMPP un dimanche matin. Elle cree `index.php`, ecrit `<?php echo "Bonjour"; ?>` et voit le texte dans son navigateur.

A quoi ca ressemble ?

```
<?php
$nom = "Lea";
echo "Salut $nom, bienvenue en PHP !";
```



Nora cree un formulaire de contact.

Petite histoire

Rasmus Lerdorf cree PHP pour gerer sa page personnelle (Personal Home Page). Le langage evolue : PHP 5 apporte l'objet, PHP 7 la performance, PHP 8 les types stricts et les attributs.

En vrai

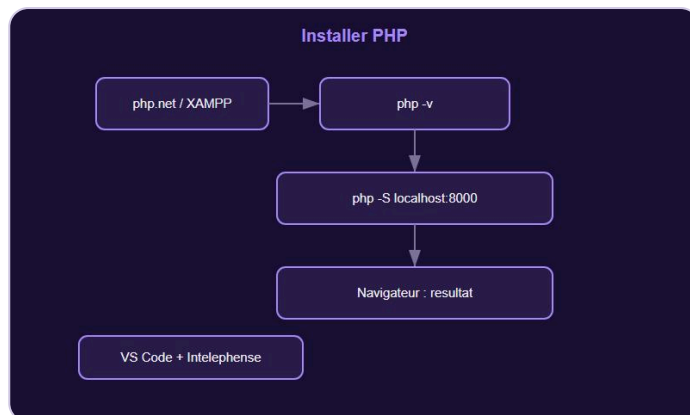
Sam cree un formulaire de contact avec PHP. Max developpe une boutique en ligne avec Laravel. Nora gere un blog WordPress et modifie les themes en PHP.

> **Piege** - PHP a une reputation de langage "ancien". En realite, PHP 8+ est moderne, type et performant.

A retenir

- PHP est cote serveur, genere du HTML dynamique.
- 77% du web tourne sur PHP.
- PHP 8+ est moderne et performant.

Installer PHP



php.net / XAMPP : le duo de départ.

Ce qu'il te faut

Pour coder en PHP tu as besoin de PHP (l'interpreteur) et d'un serveur web local. La solution la plus simple : XAMPP (Windows/Mac) ou le serveur integre de PHP.

Option 1 : serveur integre (recommande pour debuter)

1. 1. Installe PHP depuis php.net (ou via XAMPP/Laragon).
2. 2. Ouvre un terminal et tape `php -v`.
3. 3. Cree un dossier `mon-projet` avec un fichier `index.php`.
4. 4. Lance : `php -S localhost:8000`.
5. 5. Ouvre `http://localhost:8000` dans ton navigateur.

> **Astuce DanielCraft** - Le serveur integre est parfait pour apprendre. Pas besoin d'Apache ni Nginx au debut.

Option 2 : XAMPP

XAMPP installe PHP, Apache, MySQL et phpMyAdmin d'un coup. Pratique pour les projets avec base de donnees.

Installer VS Code

Telecharge VS Code. Installe l'extension "PHP Intelephense" pour l'auto-completion et la detection d'erreurs.

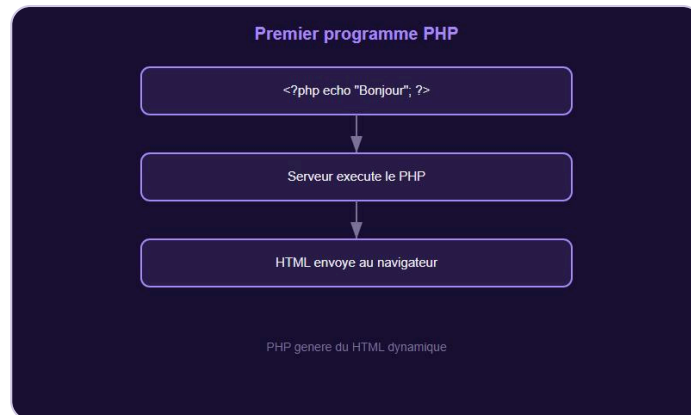
Petite histoire

Max installe PHP avec Laragon en 3 clics. Il cree `test.php`, ecrit `<?php phpinfo(); ?>`, ouvre le navigateur et voit toute la configuration PHP.

A retenir

- `php -v` pour verifier l'installation.
- `php -S localhost:8000` pour le serveur integre.
- XAMPP pour un environnement complet.

Premier programme



echo : ta premiere ligne PHP.

Hello World en PHP

```
<?php
echo "Bonjour le monde !";
```

Cree un fichier `bonjour.php`, lance `php bonjour.php` dans le terminal (ou via le serveur integre dans le navigateur).

Les balises PHP

```
<!DOCTYPE html>
<html>
<body>
  <h1><?php echo "Titre dynamique"; ?></h1>
  <p>Texte statique HTML</p>
</body>
</html>
```

PHP s'integre dans le HTML avec `<?php ... ?>`. Le serveur execute le PHP et envoie du HTML au navigateur.

echo vs print

```
echo "Bonjour";      // Plus courant
echo "A", " ", "B";  // Accepte plusieurs arguments
print("Bonjour");    // Retourne 1 (rarement utile)
```

> **Astuce DanielCraft** - Utilise `echo` partout. C'est la convention.

Les commentaires

```
// Commentaire sur une ligne
# Aussi un commentaire
/* Commentaire
   sur plusieurs lignes */
```

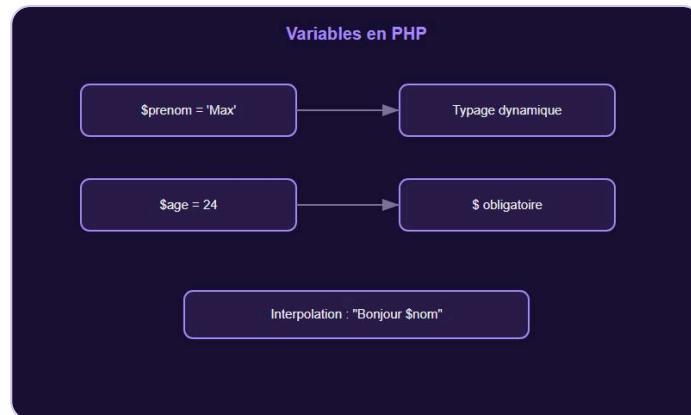
Petite histoire

Nora cree `index.php` avec un titre dynamique qui affiche la date du jour. Elle ouvre le navigateur et voit le resultat changer chaque jour.

A retenir

- `<?php ... ?>` pour le code PHP.
- `echo` pour afficher.
- PHP genere du HTML envoye au navigateur.

Les variables



\$ obligatoire, typage dynamique.

Declarer une variable

En PHP, toute variable commence par `$`. Pas besoin de déclarer le type : PHP est à typage dynamique.

```

$prenom = "Max";
$age = 24;
$prix = 19.99;
$actif = true;
  
```

Regles de nommage

- Commence par `$` suivi d'une lettre ou underscore.
- Sensible à la casse : `$nom` et `$Nom` sont différents.
- Convention : camelCase (`$monScore`) ou snake_case (`$mon_score`).

> **Astuce DanielCraft** - En PHP, `$` est obligatoire. Sans lui, c'est une constante ou un mot-cle.

Affichage

```

$ville = "Lyon";
echo "Je vis à $ville";           // Interpolation (guillemets doubles)
echo 'Je vis à ' . $ville;       // Concatenation (point)
echo "Prix : {$prix} EUR";       // Accolades pour les expressions
  
```

Constantes

```

define("TVA", 0.20);
const PAYS = "France";
echo TVA;    // 0.2
echo PAYS;   // France
  
```

Petite histoire

Sam stocke `$salaire = 1800` et `$loyer = 650`. Il affiche `$salaire - $loyer`. PHP répond 1150.

Erreur classique

```
echo $prnom; // Warning : variable non definie (faute de frappe)
```

PHP affiche un warning mais continue. Active `error_reporting(E_ALL)` pour tout voir.

A retenir

- `$` obligatoire devant chaque variable.
- Guillemets doubles pour l'interpolation.
- Point `.` pour la concatenation.

Les types de donnees



int, float, string, bool, array.

Les types de base

Type	Exemple	Usage
int	42	Entier
float	3.14	Decimal
string	"Bonjour"	Texte
bool	true / false	Vrai ou faux
array	[1, 2, 3]	Tableau
null	null	Absence de valeur

Verifier un type

```
$age = 25;
echo gettype($age);    // integer
var_dump($age);        // int(25) - plus detaille
echo is_int($age);     // 1 (true)
```

Conversion

```
$texte = "42";
$nombre = (int) $texte;    // Cast explicite
$nombre = intval($texte);  // Fonction
$flottant = (float) "3.14";
```

> **Astuce DanielCraft** - PHP fait beaucoup de conversions implicites (type juggling). C'est pratique mais peut surprendre. Utilise `===` pour comparer sans conversion.

Operations

```
$a = 10;  
$b = 3;  
echo $a + $b;    // 13  
echo $a / $b;    // 3.333...  
echo $a % $b;    // 1  
echo $a ** $b;    // 1000 (puissance)  
echo intdiv($a, $b); // 3 (division entiere)
```

Petite histoire

Nora compare `"0" == false`. PHP repond `true` (type juggling). Elle utilise `===` et obtient `false`. Lecon apprise.

A retenir

- `var_dump()` pour debuguer les types.
- `===` pour comparer sans conversion.
- `(int)`, `(string)` pour le cast explicite.

Les conditions



if / match / ?? null coalescing.

if / else

```
$age = 17;
if ($age >= 18) {
    echo "Majeur";
} else {
    echo "Mineur";
}
```

if / elseif / else

```
$note = 14;
if ($note >= 16) {
    echo "Tres bien";
} elseif ($note >= 12) {
    echo "Bien";
} elseif ($note >= 10) {
    echo "Passable";
} else {
    echo "Insuffisant";
}
```

Opérateur ternaire

```
$statut = ($age >= 18) ? "Majeur" : "Mineur";
```

Null coalescing

```
$nom = $_GET['nom'] ?? "Inconnu";
```

?? retourne la valeur de gauche si elle existe et n'est pas null, sinon celle de droite.

> **Astuce DanielCraft** - ?? est très utile pour les paramètres GET/POST optionnels.

Switch

```
$jour = "lundi";
switch ($jour) {
    case "lundi":
    case "mardi":
        echo "Debut de semaine";
        break;
    case "vendredi":
        echo "Presque le weekend";
        break;
    default:
        echo "Autre jour";
}
```

Match (PHP 8+)

```
$resultat = match($jour) {
    "lundi", "mardi" => "Debut de semaine",
    "vendredi" => "Presque le weekend",
    default => "Autre jour",
};
```

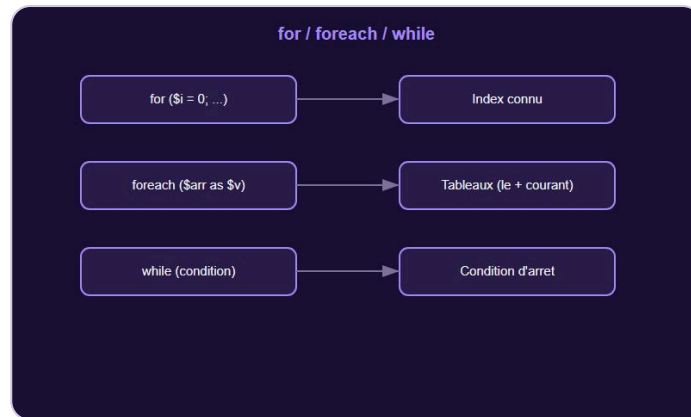
Petite histoire

Max remplace un switch de 20 lignes par un match de 5 lignes. PHP 8 simplifie le code.

A retenir

- `if / elseif / else` pour decider.
- `??` pour les valeurs par default.
- `match` (PHP 8+) remplace souvent `switch`.

Les boucles



for / foreach / while.

for

```
for ($i = 0; $i < 5; $i++) {
    echo $i . " ";
}
// 0 1 2 3 4
```

foreach

```
$fruits = ["pomme", "banane", "cerise"];
foreach ($fruits as $fruit) {
    echo $fruit . "\n";
}
```

Avec la cle :

```
foreach ($fruits as $index => $fruit) {
    echo "$index: $fruit\n";
}
```

while

```
$compteur = 0;
while ($compteur < 3) {
    echo $compteur . " ";
    $compteur++;
}
```

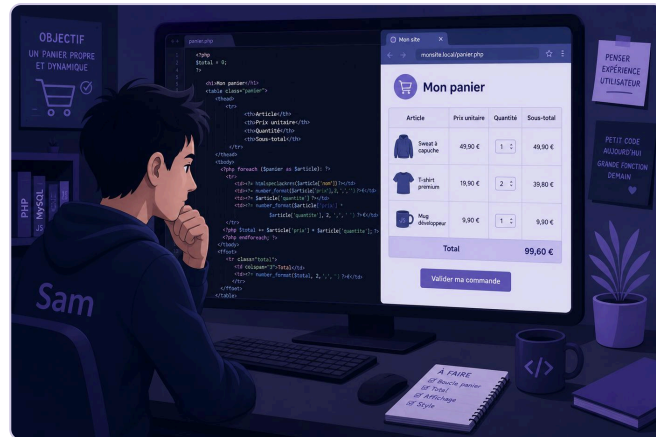
do-while

```
do {
    $saisie = readline("Un nombre > 0 : ");
} while ($saisie <= 0);
```

> **Astuce DanielCraft** - `foreach` est le plus utilise en PHP. Il parcourt tableaux et objets naturellement.

break et continue

```
for ($i = 0; $i < 10; $i++) {  
    if ($i === 5) break;  
    if ($i % 2 === 0) continue;  
    echo $i . " "; // 1 3  
}
```



Sam affiche un panier avec foreach.

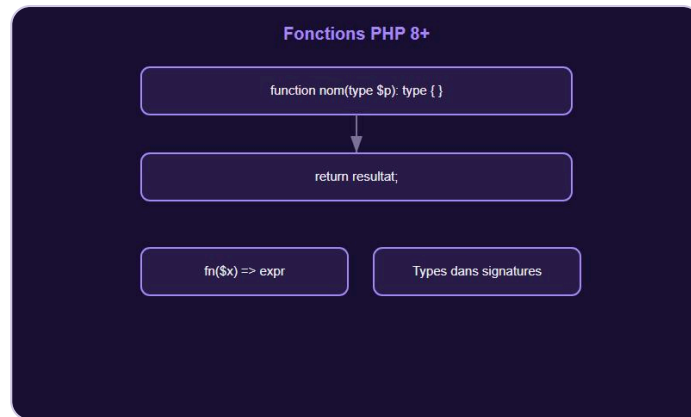
Petite histoire

Sam affiche les articles d'un panier avec `foreach`. Index, nom, prix : tout en 3 lignes.

A retenir

- `foreach` pour les tableaux (le plus courant).
- `for` quand tu connais le nombre d'iterations.
- `while` pour une condition d'arrêt.

Les fonctions



Types dans signatures, fn() fleche.

Declarer une fonction

```
function saluer(string $prenom): void {
    echo "Bonjour $prenom !\n";
}

saluer("Nora");
```

Retourner une valeur

```
function additionner(int $a, int $b): int {
    return $a + $b;
}

$resultat = additionner(3, 7);
echo $resultat; // 10
```

Parametres optionnels

```
function presenter(string $nom, string $langue = "français"): string {
    return "$nom parle $langue";
}

echo presenter("Lea"); // Lea parle français
echo presenter("Tom", "anglais"); // Tom parle anglais
```

Types de retour (PHP 8+)

```
function diviser(float $a, float $b): float|false {
    if ($b == 0) return false;
    return $a / $b;
}
```

> **Astuce DanielCraft** - Depuis PHP 8, utilise les types dans les signatures. Ça documente et protège.

Fonctions anonymes et fleches

```
$doubler = fn($x) => $x * 2;  
echo $doubler(5); // 10  
  
$filtrer = array_filter([1,2,3,4,5], fn($n) => $n > 3);  
// [4, 5]
```

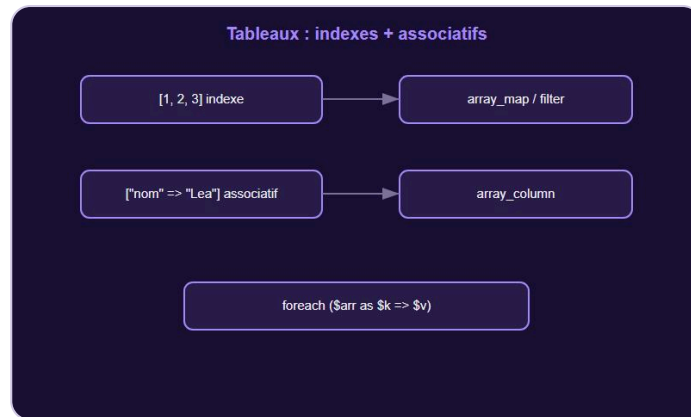
Petite histoire

Max ecrit 3 fois le meme calcul de TVA. Sam lui montre `calculerTtc($prixHt)`. Le code passe de 15 lignes a 5.

A retenir

- `function nom(params): typeRetour { ... }`
- Types dans les signatures (PHP 8+).
- `fn($x) => expr` pour les fonctions fleches.

Les tableaux



Indexes et associatifs.

Tableaux indexes

```

$fruits = ["pomme", "banane", "cerise"];
echo $fruits[0]; // pomme
echo count($fruits); // 3

```

Ajouter et supprimer

```

$fruits[] = "kiwi"; // Ajoute a la fin
array_push($fruits, "mangue");
array_splice($fruits, 1, 1); // Supprime l'index 1
unset($fruits[0]); // Supprime l'index 0

```

Tableaux associatifs

```

$personne = [
    "nom" => "Lea",
    "age" => 28,
    "ville" => "Bordeaux",
];
echo $personne["nom"]; // Lea

```

Parcourir

```

foreach ($personne as $cle => $valeur) {
    echo "$cle: $valeur\n";
}

```

Fonctions utiles

Fonction	Role
<code>count()</code>	Nombre d'elements
<code>in_array()</code>	Verifie la presence
<code>array_keys()</code>	Les cle
<code>array_values()</code>	Les valeurs
<code>sort()</code>	Trie (modifie)
<code>array_map()</code>	Applique une fonction
<code>array_filter()</code>	Filtre les elements

> **Astuce DanielCraft** - En PHP, les tableaux servent de liste ET de dictionnaire. Un seul type pour tout.

Petite histoire

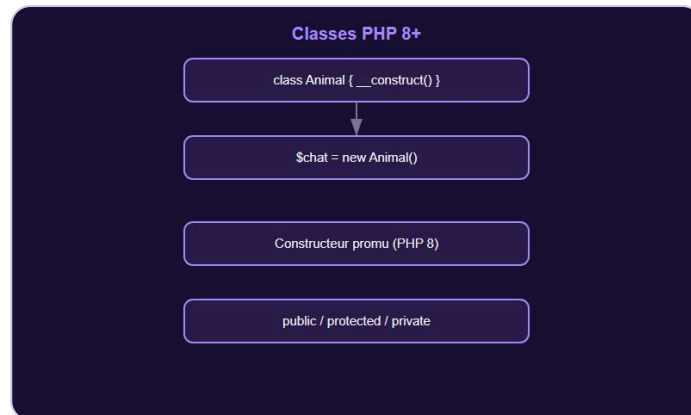
Nora stocke les prix dans un tableau associatif. Elle calcule le total avec

```
array_sum(array_values($prix)).
```

A retenir

- `[]` pour creer, `$tab[]` pour ajouter.
- `=>` pour les paires cle-valeur.
- `foreach` pour parcourir.

Classes et objets



Classes PHP 8+ : constructeur promu.

C'est quoi une classe ?

```

class Animal {
    public string $nom;
    public int $age;

    public function __construct(string $nom, int $age) {
        $this->nom = $nom;
        $this->age = $age;
    }

    public function sePresenter(): string {
        return "Je suis {$this->nom}, {$this->age} ans.";
    }
}

$chat = new Animal("Felix", 3);
echo $chat->sePresenter();
  
```

Constructeur promu (PHP 8+)

```

class Personne {
    public function __construct(
        public readonly string $nom,
        public readonly int $age,
    ) {}
}

$p = new Personne("Lea", 28);
echo $p->nom;
  
```

> **Astuce DanielCraft** - Les constructeurs promus (PHP 8) réduisent le boilerplate. Utilisez-les.

Visibilité

- **public** : accessible partout.
- **protected** : accessible dans la classe et ses enfants.
- **private** : accessible uniquement dans la classe.

Heritage

```
class Chat extends Animal {  
    public function miauler(): string {  
        return "{$this->nom} fait Miaou !";  
    }  
}
```

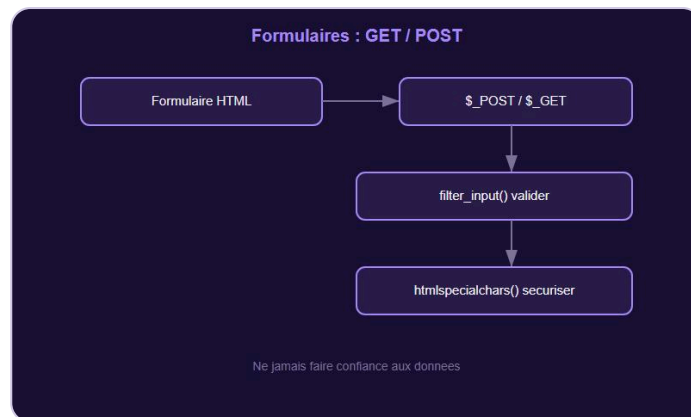
Petite histoire

Max modele un `Produit` avec nom et prix. Il cree un tableau d'objets et calcule le total avec `array_sum(array_map(...))`.

A retenir

- `class` + `new` pour creer des objets.
- `$this->` pour acceder aux proprietes.
- Constructeur promu (PHP 8+) pour simplifier.

Les formulaires



GET / POST / validation / securite.

Recevoir des donnees

PHP excelle dans le traitement de formulaires HTML.

```
<form method="POST" action="traiter.php">
  <input type="text" name="prenom">
  <button type="submit">Envoyer</button>
</form>
```

```
// traiter.php
$prenom = $_POST['prenom'] ?? '';
echo "Bonjour $prenom !";
```

GET vs POST

Methode	Usage	Donnees visibles
GET	Recherche, filtres	Oui (dans l'URL)
POST	Formulaires, creation	Non

Valider les entrees

```
$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
if ($email === false) {
    echo "Email invalide";
}
```

Securiser les sorties

```
$nom = htmlspecialchars($_POST['nom'] ?? '', ENT_QUOTES, 'UTF-8');
echo "Bonjour $nom";
```

> **Piege** - Ne jamais afficher une donnée utilisateur sans `htmlspecialchars()`. Sinon : faille XSS.

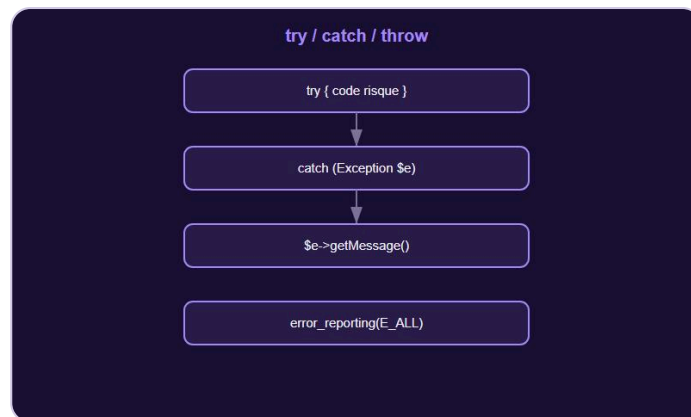
Petite histoire

Nora cree un formulaire de contact. Elle valide l'email, echappe le nom, et envoie les donnees par mail. Securise des le debut.

A retenir

- `$_GET` et `$_POST` pour recevoir les donnees.
- `filter_input()` pour valider.
- `htmlspecialchars()` pour securiser l'affichage.

Gerer les erreurs



try/catch/throw.

Les exceptions

```

try {
    $nombre = intval(readline("Un nombre : "));
    if ($nombre === 0) {
        throw new InvalidArgumentException("Zero interdit");
    }
    echo 10 / $nombre;
} catch (InvalidArgumentException $e) {
    echo "Erreur : " . $e->getMessage();
} catch (DivisionByZeroError $e) {
    echo "Division par zero !";
} finally {
    echo "\nFin du bloc.";
}
  
```

Erreurs courantes

Erreur	Cause
Warning	Variable non definie, fichier absent
Fatal error	Appel de methode sur null
TypeError	Mauvais type d'argument
ParseError	Erreur de syntaxe

Configurer le reporting

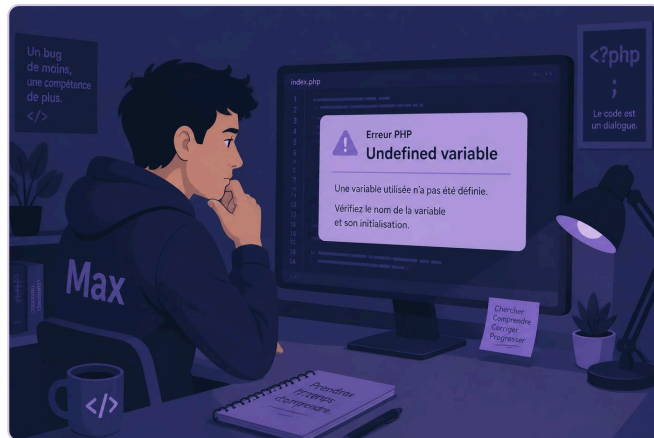
```

error_reporting(E_ALL);
ini_set('display_errors', '1');
  
```

> **Astuce DanielCraft** - Active `E_ALL` en developpement. Desactive `display_errors` en production.

Lever une exception

```
function retirer(float $solde, float $montant): float {  
    if ($montant > $solde) {  
        throw new RuntimeException("Solde insuffisant");  
    }  
    return $solde - $montant;  
}
```



Max lit un Undefined variable.

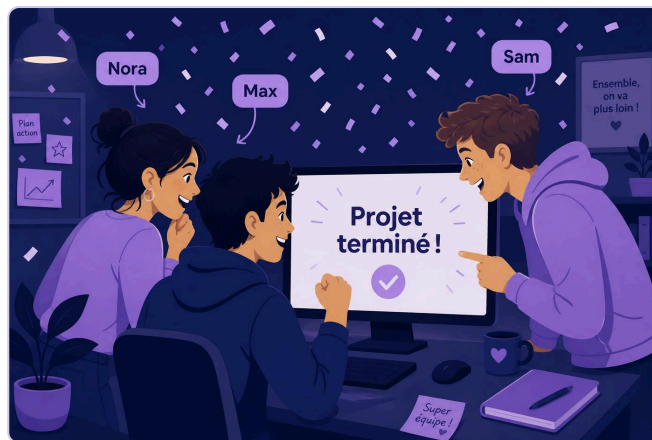
Petite histoire

Max oublie un point-virgule. PHP affiche `ParseError`. Il lit le message, corrige, relance. En PHP 8, les messages d'erreur sont clairs.

A retenir

- `try/catch/finally` pour les exceptions.
- `throw` pour lever ses erreurs.
- `error_reporting(E_ALL)` en dev.

Mini-projet : page de contacts



Nora, Max et Sam valident le projet.



Mini-projet : page de contacts.

L'objectif

On crée une page web PHP qui permet d'ajouter et lister des contacts. Les données sont stockées dans un fichier JSON. Ce projet combine formulaires, tableaux, fonctions, fichiers et HTML.

Structure

```

<?php
$fichier = 'contacts.json';

function charger(string $f): array {
    if (!file_exists($f)) return [];
    $json = file_get_contents($f);
    return json_decode($json, true) ?? [];
}

function sauvegarder(string $f, array $contacts): void {
    file_put_contents($f, json_encode($contacts, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE));
}

$contacts = charger($fichier);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

```

```

$nom = trim($_POST['nom'] ?? '');
$tel = trim($_POST['tel'] ?? '');
if ($nom !== '' && $tel !== '') {
    $contacts[] = ['nom' => $nom, 'tel' => $tel];
    sauvegarder($fichier, $contacts);
}
}
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>Contacts</title></head>
<body>
<h1>Mes contacts</h1>
<form method="POST">
    <input name="nom" placeholder="Nom" required>
    <input name="tel" placeholder="Telephone" required>
    <button type="submit">Ajouter</button>
</form>
<ul>
<?php foreach ($contacts as $c): ?>
    <li><? = htmlspecialchars($c['nom']) ?> - <? = htmlspecialchars($c['tel']) ?></li>
<?php endforeach; ?>
</ul>
</body>
</html>

```

Ce que tu apprends

- `json_encode` / `json_decode` pour la persistance.
- Traitement de formulaire POST.
- `htmlspecialchars` pour la securite.
- Syntaxe alternative `foreach ... endforeach` dans le HTML.

> **Astuce DanielCraft** - Commence par le formulaire HTML, puis ajoute le PHP progressivement.

A retenir

- Un projet = assemblage de notions.
- JSON pour persister sans base de donnees.
- Toujours echapper les sorties HTML.

Ce qu'il faut retenir



La carte des notions PHP.

Les briques essentielles

Concept	Resume
Variables	<code>\$nom</code> , typage dynamique
Types	<code>int</code> , <code>float</code> , <code>string</code> , <code>bool</code> , <code>array</code>
Conditions	<code>if/elseif/else</code> , <code>match</code> , <code>??</code>
Boucles	<code>for</code> , <code>foreach</code> , <code>while</code>
Fonctions	Types dans signatures (PHP 8+), <code>fn()</code>
Tableaux	Indexes et associatifs, <code>foreach</code>
Classes	<code>__construct</code> , visibilite, heritage
Formulaires	<code>\$_GET</code> , <code>\$_POST</code> , validation
Erreurs	<code>try/catch</code> , <code>throw</code>

La methode

1. Lis le chapitre.
 2. Tape les exemples toi-meme.
 3. Modifie et observe.
 4. Fais les ateliers.
 5. Reviens quand tu bloques.
- > **Astuce DanielCraft** - PHP a enormement de fonctions integrees. Consulte php.net regulierement.

Ce qui vient apres

- PDO et les bases de donnees.

- Sessions et cookies.
- Composer et les packages.
- Laravel ou Symfony.
- API REST en PHP.

A retenir

- Tu as les bases. PHP est vaste mais accessible.
- php.net est ta reference.

Atelier : variables et types



Atelier : variables et types.

Exercice 1 : carte d'identité

```
$prenom = "Sam";
$age = 22;
$ville = "Nantes";
echo "Je suis $prenom, $age ans, je vis a $ville.\n";
```

Exercice 2 : convertisseur EUR -> USD

```
$euros = (float) readline("Montant en EUR : ");
$dollars = $euros * 1.08;
echo number_format($euros, 2) . " EUR = " . number_format($dollars, 2) . " USD\n";
```

Exercice 3 : comparaison stricte

```
var_dump("0" == false); // true (type juggling)
var_dump("0" === false); // false (comparaison stricte)
var_dump(0 == "abc"); // true en PHP 7, false en PHP 8
```

Defi : temperature C -> F

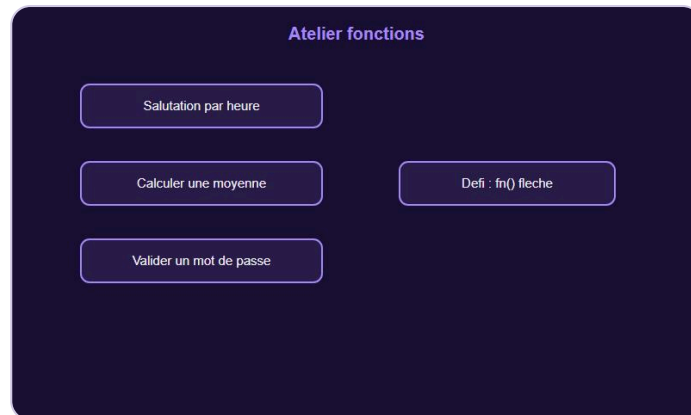
```
$c = (float) readline("Celsius : ");
$f = $c * 9 / 5 + 32;
echo "$c°C = " . number_format($f, 1) . "°F\n";
```

> **Astuce DanielCraft** - `number_format()` pour un affichage propre des nombres.

A retenir

- `$` devant chaque variable.
- `===` pour la comparaison stricte.
- `number_format()` pour formater.

Atelier : fonctions



Atelier : fonctions typees.

Exercice 1 : salutation

```
function saluer(string $nom, int $heure): string {
    if ($heure < 12) return "Bonjour $nom !";
    if ($heure < 18) return "Bon apres-midi $nom !";
    return "Bonsoir $nom !";
}

echo saluer("Lea", 9) . "\n";
echo saluer("Max", 20) . "\n";
```

Exercice 2 : moyenne

```
function moyenne(array $notes): float {
    if (count($notes) === 0) return 0;
    return array_sum($notes) / count($notes);
}

echo moyenne([14, 16, 11, 18]); // 14.75
```

Exercice 3 : mot de passe valide

```
function mdpValide(string $mdp): bool {
    return strlen($mdp) >= 8 && preg_match('/\d/', $mdp);
}

var_dump(mdpValide("abc")); // false
var_dump(mdpValide("PHP8rocks!")); // true
```

Exercice 4 : fonction fleche

```
$ttc = fn(float $ht) => $ht * 1.20;
echo $ttc(100); // 120
```

> **Astuce DanielCraft** - `array_sum()`, `count()`, `preg_match()` : PHP a des fonctions pour presque tout.

A retenir

- Types dans les signatures pour la clarte.
- Fonctions fleches `fn()` pour les expressions courtes.
- La bibliotheque standard PHP est enorme.

Atelier : tableaux



Atelier : tableaux et `array_*`.

Exercice 1 : filtrer les pairs

```
$nombres = [3, 8, 1, 12, 7, 4, 15, 2];
$pairs = array_filter($nombres, fn($n) => $n % 2 === 0);
print_r(array_values($pairs));
```

Exercice 2 : compteur de mots

```
$phrase = "le chat dort le chat mange le chat joue";
$mots = explode(" ", $phrase);
$compteur = array_count_values($mots);
print_r($compteur);
```

Exercice 3 : transformer un tableau

```
$noms = ["lea", "max", "sam"];
$majuscules = array_map('ucfirst', $noms);
print_r($majuscules); // [Lea, Max, Sam]
```

Exercice 4 : panier d'achat

```
$panier = [
    ["nom" => "Pain", "prix" => 1.20],
    ["nom" => "Lait", "prix" => 0.95],
    ["nom" => "Oeufs", "prix" => 2.50],
];
$total = array_sum(array_column($panier, "prix"));
echo "Total : " . number_format($total, 2) . " EUR\n";
```

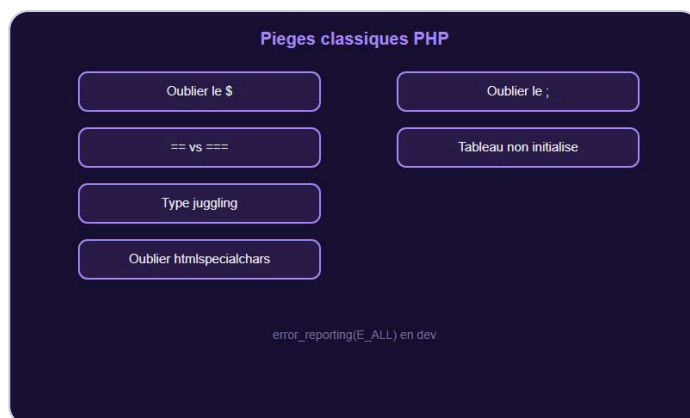
> **Astuce DanielCraft** - `array_column()` extrait une colonne d'un tableau multidimensionnel. Très pratique.

A retenir

- `array_filter()`, `array_map()`, `array_column()`.
- `array_count_values()` pour compter les occurrences.

- `explode()` pour decouper une chaine en tableau.

Erreurs classiques en PHP



Pieges : \$, ==, htmlspecialchars.

1. Oublier le \$ devant une variable

```
echo nom; // Constante "nom" non definie
echo $nom; // Correct
```

2. Confondre = et ==

```
if ($x = 5) { } // Assigne ! Toujours vrai.
if ($x == 5) { } // Compare
if ($x === 5) { } // Compare strictement
```

3. Type juggling surprenant

```
var_dump("0" == false); // true
var_dump("" == false); // true
var_dump("abc" == 0); // true en PHP 7 !
```

Utilise `===` pour eviter les surprises.

4. Oublier htmlspecialchars

```
echo $_GET['nom']; // Faille XSS !
echo htmlspecialchars($_GET['nom'], ENT_QUOTES, 'UTF-8'); // Securise
```

5. Tableau non initialise

```
$items[] = "test"; // Warning si $items n'existe pas
$items = []; // Initialise d'abord
$items[] = "test";
```

6. Point-virgule oublie

```
echo "Bonjour" // ParseError !
echo "Bonjour"; // Correct
```

> **Astuce DanielCraft** - Active `error_reporting(E_ALL)` et `display_errors` en dev. En production, logge les erreurs.

A retenir

- `$` obligatoire, `;` obligatoire.
- `===` plutôt que `==`.
- `htmlspecialchars()` pour toute sortie.

Bonnes pratiques



`strict_types`, PSR-12, PHPStan.

Style de code

- **PSR-12** : le standard de style PHP.
- camelCase pour les methodes et variables.
- PascalCase pour les classes.
- 4 espaces pour l'indentation.

Typage strict

```
declare(strict_types=1);

function calculerTtc(float $ht, float $tva = 0.20): float {
    return $ht * (1 + $tva);
}
```

`declare(strict_types=1)` en haut de chaque fichier force la verification des types.

> **Astuce DanielCraft** - Active toujours `strict_types`. Ca detecte les bugs tot.

Securite

- `htmlspecialchars()` pour l'affichage.
- `filter_input()` pour la validation.
- Requetes preparees (PDO) pour les bases de donnees.
- Ne jamais faire confiance aux donnees utilisateur.

Organisation

```
// 1. declare + namespace
declare(strict_types=1);
namespace App;

// 2. use
use App\Service\MailService;

// 3. Classe
class ContactController {
    // ...
}
```

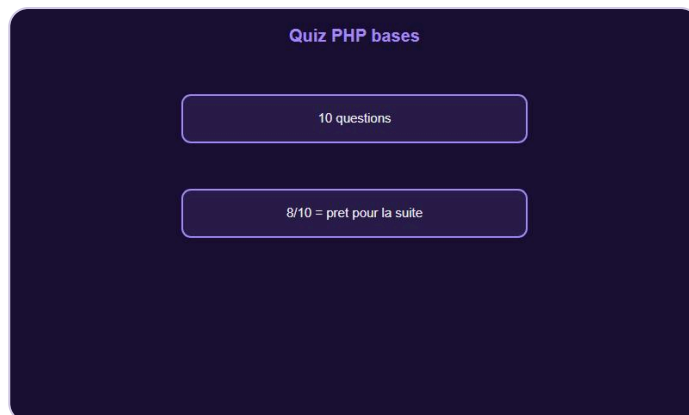
Outils

- **Composer** : gestionnaire de dependances.
- **PHPStan / Psalm** : analyse statique.
- **PHP CS Fixer** : formatage automatique.

A retenir

- `declare(strict_types=1)` dans chaque fichier.
- PSR-12 pour le style.
- PHPStan pour detecter les erreurs avant l'execution.

Quiz PHP - Les bases



Quiz PHP bases.

Teste tes connaissances

Q1. Comment commence une variable en PHP ?

A) var B) \$ C) let D) @

Q2. Quel operateur compare sans conversion de type ?

A) == B) === C) != D) <>

Q3. `echo` sert a :

A) Lire une variable B) Afficher du texte C) Declarer une fonction D) Creer un tableau

Q4. Comment parcourir un tableau associatif ?

A) for B) foreach C) while D) map

Q5. `$_POST` contient :

A) Les cookies B) Les donnees d'un formulaire POST C) Les sessions D) Les variables d'environnement

Q6. Comment securiser l'affichage d'une donnee utilisateur ?

A) echo \$data B) htmlspecialchars(\$data) C) strip_tags(\$data) D) addslashes(\$data)

Q7. Le constructeur d'une classe s'appelle :

A) `__init__` B) `__construct` C) constructor D) init

Q8. `fn($x) => $x * 2` est :

A) Une methode B) Une fonction anonyme fleche C) Un decorateur D) Un callback

Q9. `??` en PHP est :

A) L'operateur ternaire B) Le null coalescing C) La comparaison stricte D) Le spaceship operator

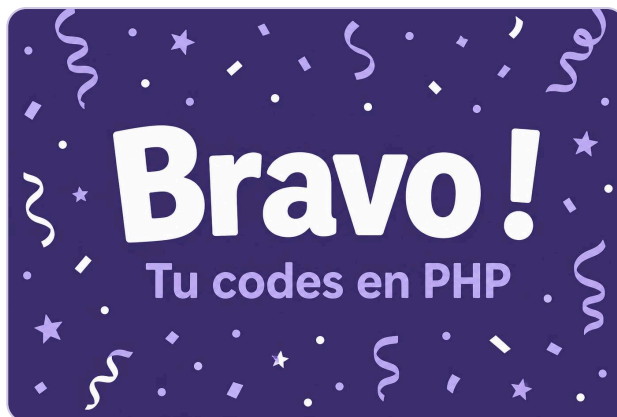
Q10. `declare(strict_types=1)` sert a :

A) Activer le mode debug B) Forcer la verification des types C) Desactiver les warnings D) Activer le cache

Reponses

1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

> **Astuce DanielCraft** - 8/10 ou plus ? Tu es pret pour PHP intermediaire.

Bravo !

Bravo. Tu codes en PHP.

Tu as termine PHP - Les bases

Félicitations. Tu sais écrire des scripts PHP, créer des pages dynamiques, traiter des formulaires et gérer des erreurs. Tu maîtrises les variables, les types, les conditions, les boucles, les fonctions, les tableaux, les classes et la sécurité de base.

Ce que tu as construit

- Un premier programme dynamique.
- Des fonctions typées et réutilisables.
- Une page de contacts complète.
- Des exercices sur chaque notion.

La suite avec DanielCraft

Le livre **PHP - Intermédiaire** couvrira :

- PDO et les bases de données.
- Sessions et cookies.
- Composer et les packages.
- Architecture MVC.
- Introduction à Laravel.

> **Astuce DanielCraft** - Choisis un petit projet web : un formulaire de contact, un blog simple, un outil d'inventaire. Construis pour apprendre.

Un dernier conseil

PHP est partout sur le web. Chaque site WordPress, chaque boutique PrestaShop, chaque application Laravel tourne sur PHP. Tu as les bases d'un langage qui propulse le web.

Tu as les bases. Maintenant, déploie.

Tu as les briques. Maintenant, construis.